

Oxide AI — project handoff

Snapshot: 17 September 2026 UTC / 16 September 2026 Pacific. Owner: Spart / sparticle62ops.

Private recovery document. Do not publish the complete recovery archive, private transcripts, or corpus files to GitHub. This PDF is the map; the accompanying archive contains the actual source, trained weights, experiment records, and portable Git history.

1. Start here

The CPU training, checkpoint, BPE tokenization, evaluation, and generation pipeline works on the known-good baseline. **Broadly coherent and logical language generation has NOT been achieved.** Lower validation loss is real, but the unselected outputs still repeat, drift off-topic, and contain malformed phrases.

The active work separates a reusable continuous processing block from the vocabulary endpoints. It is not yet a completed multilayer architecture comparison. Never describe the planned depth2/depth4 study as completed training.

The user's required order is binding: first correct and train the model until its language works reproducibly; only then revamp the CLI; only after that implement full WGPU detection and training/inference support, with an interactive hardware choice. Existing historical backend code/dependencies are not evidence of working GPU execution. Do not begin UI redesign or new GPU implementation now.

Publication status

Repository: <https://github.com/Sparticle62ops/oxide-ai>

A fresh remote check found main at 817fc6fec001840d49f4f56352e7e767c1bf8431, with only main and stabilization/oxide-repair listed. The original stabilization is merged. **The newer continuation and extraction changes are not confirmed pushed.** Do not confuse a local commit or a saved bundle with GitHub publication.

The attached integration can create commits but exposes no author/committer controls. The user requires BOTH identities to be sparticle62ops <sparticle62ops@users.noreply.github.com>. An existing-connection picker is open for the user's GitHub API connection. Do not silently replace the identities with the app account. Fresh status is in publication/remote-handoff-check.json.

2. Files and recovery layout

All paths below are relative to the thread's work directory, unless stated otherwise. Current persistent root:

`/tasklet/threads/a_rkbhvdvaz9a63sqhx7mr/work`

- `project/`: known-good continuation baseline; leave untouched until the extracted model is accepted.
- `depth-project/`: current experimental source and tests. This is the directory to continue editing.
- `depth-evidence/stage-a/`: earlier inference-only extraction evidence and tested archive. That earlier revision passed 34 tests, but did not include the later continuous backward extraction.
- `depth-evidence/stage-a-complete/`: latest extraction audit, golden BPE files, new acceptance tests and build logs.
- `experiments/`: training records, initial/per-epoch checkpoints, fixed generations, and diagnostics.
- `publication/`: attributed continuation Git bundle, mailbox patches, remote checks, and publication records.
- `scripts/`: reproducible build, comparison, packaging, and recovery helpers.
- `oxide-ai-handoff/`: original imported project handoff, historical source/evidence, corpus split, and private prior context.
- `experiments/WIDTH-DEPTH-PROTOCOL.md`: the frozen comparative evaluation plan.
- `depth-evidence/STAGE-B-IMPLEMENTATION.md`: proposed stack implementation and acceptance contract; design, not completed implementation.

The environment under `/tmp` is ephemeral. A reset already destroyed one toolchain/build cache in this session. Never assume a background job, local compiler, or temporary clone survived. Persistent source and evidence under the work directory survived. An unfinished build log is not a passing test run.

3. Verified training evidence

The reported runs used width64, depth1, BPE vocabulary2048, state8, memory-key16, memory capacity32, chunk64, seed42. The default width256 configuration was NOT the source of the reported language samples.

- Initialization: validation NLL 7.9479712 nats/BPE transition; perplexity 2829.8279677; next-token accuracy 0.02644%.
- One full pass: 840,951 scored training transitions; validation NLL 5.1704721; perplexity 175.9978987; accuracy 11.5541%.
- Longer control epoch1: validation NLL 5.1661181; perplexity 175.2332734; accuracy 11.60166%.
- Longer control epoch2: validation NLL 4.9017792; perplexity 134.5289243; accuracy 14.07354%.
- Longer control epoch3: validation NLL 4.7431985; perplexity 114.8008044; accuracy 15.68645%.

The longer control completed **three**, not four, passes: 2,522,853 scored transitions and 9,888 updates. Four passes were planned. There is no evidenced epoch4 checkpoint or final.pssa for this run. The prior incorrect completion claim is retained separately and marked superseded.

The test split has not been evaluated. These are held-out-article validation results in BPE transition units, not a test of reasoning. For example, a completion of “The purpose of a scientific experiment is” degenerates into “the sodium and the state...” and repeated United States fragments. Preserve every fixed generation; do not select only plausible phrases.

Primary evidence: experiments/bpe-long-pass-01/epochs/metrics.json, its experiment/validation records, and generations.json. Earlier full-pass and pilot results remain in their respective directories.

4. Current extraction and bugs found

PSSAContinuousBlockV2 now owns affine RMS normalization, SSM matrices and carry, the detached episodic bank, adapter, MLP, and continuous activation/adjoint buffers. PSSALayerV2 owns the shared embedding/head, token/target/logit/loss tape, tokenizer metadata and one optimizer clock. The block receives raw continuous rows and returns raw-input adjoints from arbitrary supplied output adjoints.

The audit found no concrete algebra, random-draw-order, optimizer-order, or V7 serialization-order drift. That is a source audit, not a replacement for executable parity tests.

Fixed during this continuation:

- An allocation regression test still referenced the former model.memory field rather than model.block.memory, preventing full test compilation.
- The earlier continuous-inference test pre-normalized input, although the new block now owns normalization. It now passes raw embedding rows and retains its exact output/state assertions.
- Checkpoint allocation accounting missed four new chunk-by-width buffers plus one width-sized inference buffer. Checked accounting was increased by $(4 * \text{chunk_len} * \text{d_latent} + \text{d_latent}) * 4$ bytes. This fixes the loader's allocation-budget estimate without changing V7 bytes.

New tests cover arbitrary external-adjoint finite differences, input derivatives, representative norm/SSM/query/memory/adapter/MLP parameters with nonempty detached memory, VJP scaling/additivity, split-chunk carry and detachment, endpoint-only embedding scattering, vocabulary-independent block storage, and actual counted zero-allocation paths. Tests must pass before these are treated as proven properties.

Latest acceptance status

The corrected extraction passed the complete release suite: **41 tests, 0 failures, exit0**. This includes the original regression tests and 7 new continuous-block/allocation tests. The external-adjoint finite-difference budget was tightened to 5e-6 absolute plus 2% relative; the suite passed again. The small nonempty-memory Qh derivative is approximately -4.9934e-5 analytically versus -4.9826e-5 numerically, well above the absolute tolerance.

Golden compatibility also passed: initial, epoch1, epoch2 and final tiny-BPE checkpoint files are all byte-identical to the old executable. The loaded bank contains 4 entries. Two subsequent optimizer updates from that populated-memory checkpoint are byte-identical to the original baseline, ending at update194. Greedy and sampled generation (temperatures0 and 0.8) match byte-for-byte. Evidence is in golden-bpe-parity.json, next-update-parity.sha256/.exit, and generation-parity.sha256/.exit.

Stage A is accepted for the checked compatibility and derivative properties. Genuine depth2/depth4 stacking and the controlled width/depth training study have NOT yet been implemented or run. The 41-test result does not establish broad language coherence.

The authoritative full-suite record is depth-evidence/stage-a-complete/acceptance-tests.log with its exit-code file. Read the current log together with process status: a stale exit file can belong to a prior failed attempt. Initial environment restoration failed with exit127 before Rust was reinstalled. Rust1.90 was subsequently restored and a new release build started.

5. Reproduce validation and golden comparisons

Install Rust1.90.0 with the minimal profile if the environment has reset. Use one Cargo build job on this two-CPU/four-GB environment. Build settings are locked dependencies, release opt-level3, native CPU target, warnings denied, and explicitly disabled LTO for iteration. Do not compare performance across different build settings or hardware without labeling it.

From the work root:

```
bash scripts/validate-depth.sh
```

The helper points at the current persistent depth-project and writes durable logs. For a portable checkout, use:

```
export CARGO_BUILD_JOBS=1
export CARGO_PROFILE_RELEASE_LTO=false
export RUSTFLAGS="-Dwarnings -C target-cpu=native"
cargo +1.90.0 test --locked --release -j1
```

After the full suite passes, run `scripts/golden-parity.ts` using Bun. It reconstructs the old tiny-BPE training argument set from the recorded run metadata, runs the new executable, and compares initialization, both epoch checkpoints, and final checkpoint byte-for-byte. It refuses reuse of an existing output run directory; preserve old attempts and choose a fresh directory when rerunning.

Then run `scripts/golden-continuation.sh`. It compares two subsequent optimizer updates from the same populated-memory checkpoint against the original baseline source in a temporary clone. It does not edit `project/`. Compare generated output from the original and extracted executable as a separate final gate. Do not skip populated-memory BPE checks merely because tiny word tests pass.

6. Authorized controlled experiment

The user explicitly authorized four configurations: width64/depth1 baseline, width256/depth1 wider control, width64/depth2, and width64/depth4. Full protocol is in `experiments/WIDTH-DEPTH-PROTOCOL.md`.

Freeze the train/validation split and exact tokenizer metadata from the baseline initial checkpoint. Use four full passes per configuration, no prefix cap, same seed, ordering, document resets, hyperparameter schedule and token-weighted accumulation. Expected exposure is 3,363,804 scored transitions per configuration; verify the encoded count and hashes before training. A fresh baseline is necessary because the earlier four-pass plan only completed three.

Primary report: final-epoch held-out NLL/perplexity and accuracy; boundary-adjoint mean, population variance and RMS at fixed update indices; parameter counts; scored training transitions per second; separate artifact/evaluation overhead. Run configurations sequentially to avoid CPU contention. Equal token exposure is not equal compute or equal parameter count.

Retain all existing fixed prompts and temperatures, sampling seed1337, without rerolls or repetition penalties. Assess repetition, prompt relevance, grammaticality and logical consistency directly. One seed is a pilot, not statistical proof; replicate apparent winners before making strong architecture claims.

Only implement the stack after Stage A passes. Proposed deeper path: keep the original first block, then add continuous residual blocks with fixed scale $1/\sqrt{\text{depth}}$, independent carry/tape/memory and a shared vocabulary head. This nonzero scale starts branch gradients immediately, but is not a function-preserving expansion. Test full cross-layer derivatives, depth2/depth4 inference/training equivalence, checkpoint/optimizer continuation, allocation safety and zero-allocation hot paths before large training.

7. Safe GitHub publication and restart checklist

Existing local attributed continuation commits:

- dac403de74b810742692e21ac2282eb76baebcf1
- d27c3681055127a62a933aa20a89babbcfc20cf7

The newly tested extraction is committed locally as edea54543dadfa8eeb3f4c2815b327d8ae47d9ea, with both required identities verified. Branch: recovery/continuous-block-20260917. The complete-history recovery bundle is publication/continuous-block-recovery.bundle; Git's bundle verification succeeded. Its mbox and source patch are alongside it. This local commit has NOT been confirmed on GitHub.

To restore portable source history, clone the recovery bundle with Git and check out the recovery branch. The bundle contains its own complete history and does not depend on a live Tasklet environment. The original continuation.bundle remains available separately. The full recovery ZIP additionally contains the trained checkpoints and private experiment context that intentionally do not belong in the source repository.

1. Restore or attach the identity-preserving GitHub API connection, or push the attributed local branch from the owner's own Git client.
2. Fetch and inspect current main; do not overwrite newer upstream changes. Keep experimental changes on a review branch rather than force-pushing main.
3. Commit only reviewed source/tests and repository-safe experiment summaries, with both required identities. Keep complete recovery archives, transcripts and corpus out of the repository.
4. Push, then re-read the remote branch/commit and verify content and identity. Record URL and SHA in the handoff. Until then, publication status remains local-only.
5. Verify the recovery archive's SHA-256 and per-file checksums before relying on a downloaded copy.
6. Resume in depth-project, finish the Stage-A gates, preserve the accepted revision, then implement/validate the stack and execute the frozen comparison.

Success criterion remains unselected coherent and logical output, not merely successful compilation, lower perplexity, or a larger layer count.